Nina Pakshina

Go developer in an online retail company. LinkedIn: https://www.linkedin.com/in/nina-pakshina-228156114/ Buy me a coffee: https://www.buymeacoffee.com/ninucium

Follow writer

Go Interview Questions, Part 1: Pointers, channels, and range

Nina Pakshina Follow 6 min read · May 8, 2023

Hello everyone! My name is Nina and I work as a Go developer.

Among IT specialists, there is a practice of going to interviews even if you are not actively looking for a job. It helps to maintain your level of knowledge, learn something new, and of course, there is always a chance to find a job that you really like :) And let's not forget those who are currently actively looking for work.

Therefore, I want to open a new section that will be useful for those who are planning to interview soon, as well as for those who want to deepen their knowledge of Go (because let's be honest: the problems you encounter in interviews are quite rare in your day-to-day work).

In this series, I will be posting tricky questions and tasks from interviews that I have encountered, presented in the form of mock conversations with the interviewer.

Generated by Kandinsky 2

. . .

Pointers, channels, and range

Usually, one of the interview sessions is conducted in the form of live coding. During one of such sessions, I was given the following code in an Go online IDE:

```
package main

import (
  "fmt"
  "sync"
)

func main() {
  ch := make(chan *int, 4)
  array := []int{1, 2, 3, 4}
  wg := sync.WaitGroup{}
  wg.Add(1)
  go func() {
    for _, value := range array {
      ch <- &value
    }
  }()
  go func() {
    for value := range ch {
      fmt.Println(*value) // what will be printed here?
    }
    wg.Done()
  }()

  wg.Wait()
}
```

Overall, I was faced with everyone's favorite task of compiling code in my head.

> *Interviewer: What will happen when the code is compiled, what will the line `fmt.Println(*value)` output, and how can we fix all errors?*

When dealing with goroutines, it makes sense to immediately analyze the code for the possibility of deadlock.

On the one hand, everything looks correct:

- We wait for all goroutines to finish using `WaitGroup` and `wg.Wait()`
- We increase the wait group counter by 1 using `wg.Add(1)` before calling the first goroutine
- In the second goroutine, we decrease it by 1 after reading all data from the ch channel `wg.Done()`

But here it is necessary to remember how range works with reading data from a channel.

The point is that the `ch` channel is not closed, that's why the line `wg.Done()` in the second goroutine will never be reached.

The `for value := range ch` loop will be blocked waiting for the channel to be closed if no values are written to the channel for reading.

Therefore, the answer is: we will have a **deadlock**!

> *Interviewer: How to fix the deadlock?*

Deadlock can be fixed in several ways. The simplest and correct way is to close the channel after all writes to the channel in the first goroutine. After that, we can exit the `for value := range ch` loop in the second goroutine and decrease the `WaitGroup` counter to 0.

Here's how to fix the code so it doesn't crash:

```
package main

import (
  "fmt"
  "sync"
)

func main() {
  ch := make(chan *int, 4)
  array := []int{1, 2, 3, 4}
  wg := sync.WaitGroup{}
  wg.Add(1)
  go func() {
    for _, value := range array {
      ch <- &value
    }
    // Close channel to reach wg.Done() line.
    close(ch)
  }()
  go func() {
    for value := range ch {
      fmt.Println(*value)
    }
    wg.Done()
  }()
  wg.Wait()
}
```

There is an alternative way to solve the deadlock issue:

```
package main

import (
  "fmt"
  "sync"
)

func main() {
  ch := make(chan *int, 4)
  array := []int{1, 2, 3, 4}
  wg := sync.WaitGroup{}
  // Add the number of works equal to the number of array elements.
  wg.Add(len(array))
  go func() {
    for _, value := range array {
      ch <- &value
    }
  }()
  go func() {
    for value := range ch {
      fmt.Println(*value)
      // Decrement the waitgroup counter with each iteration.
      wg.Done()
    }
  }()

  wg.Wait()
}
```

Here we increased the counter by the length of the `array`, and then in the second goroutine, while iterating through the channel with wg.Done() in the loop `for value := range ch`, we decreased the counter to 0.

After that, the counter in `WaitGroup` will be reset to 0 and the program will complete execution.

> *Interviewer: And what will happen if we don't close the channel?*

In our case, if we don't close the channel, nothing bad will happen, since the program will immediately terminate and all memory will be freed. However, it is important to remember that if a channel is not closed in a long-running application, data leakage may occur and memory will be held until the program terminates.

> *Interviewer: Now that our code is working, what will be outputted by fmt.Println(*value)?*

It will output 4, 4, 4, 4.

> *Interviewer: Why does this happen?*

It's all about how range works in Golang.

For optimization purposes in Go, new variables for the index and value will not be created for each iteration. Instead, two variables are created that will be used to store the index and value in each iteration of the range loop.

In the buffered channel `ch <- &value`, we pass the same pointer, and since the channel is buffered, we will pass all 4 elements to it at once.

By the time we read the value from the channel and print the result, the first range loop has already completed and the value of the last element (4) will be at the address of the value. Therefore, `fmt.Println(*value)` will output `4, 4, 4, 4` (you can read more about how this is implemented in the language specification).

> *Interviewer: And how can we fix this?*

To fix this, it is enough to not pass a pointer to the value directly, but to create a new variable `v` each time and copy the value to it in each iteration, and then pass its address to the channel.

Here is the corrected code:

```
package main

import (
  "fmt"
  "sync"
)

func main() {
  ch := make(chan *int, 4)
  array := []int{1, 2, 3, 4}
  wg := sync.WaitGroup{}
  wg.Add(len(array))
  go func() {
    for _, value := range array {
      // Temporary variable v that has new memory address every iteration.
      v := value
      ch <- &v
    }
  }()
  go func() {
    for value := range ch {
      fmt.Println(*value)
      wg.Done()
    }
  }()

  wg.Wait()
}
```

After making this correction, `fmt.Println(*value)` will output `1, 2, 3, 4.`

Spoiler alert: By the way, there is a simpler variation of this problem on interview and you know how to solve it now:

```
package main

import "fmt"

func main() {
  array := []int{1, 2, 3, 4}
  newArray := make([]*int, 4)
  for i, value := range array {
    newArray[i] = &value
  }
  for _, value := range newArray {
    fmt.Println(*value)
  }
}
```

Bonus question

> *Interviewer: Let's go back to the original code that crashed with a deadlock. What will happen if we start another goroutine with an infinite for loop? Here's the code:*

```
package main

import (
  "fmt"
  "sync"
)

func main() {
  ch := make(chan *int, 4)
  array := []int{1, 2, 3, 4}
  wg := sync.WaitGroup{}
  wg.Add(1)
  go func() {
    for _, value := range array {
      ch <- &value
    }
  }()
  go func() {
    for value := range ch {
      fmt.Println(*value)
    }
    wg.Done()
  }()

  // New goroutine is run.
  go func() {
    var a int
    for {
      a++
    }
  }()

  wg.Wait()
}
```

> *Interviewer: Will there be a deadlock here?*

No. Deadlock occurs when all goroutines in the program are blocked and cannot continue their work, for example, waiting for access to shared resources or waiting for each other. For example, if one goroutine is blocked on reading from an empty channel, and another goroutine is blocked on writing to the same channel, this will lead to a deadlock. Both goroutines will be waiting for each other and will not be able to continue executing their tasks. If one goroutine is not blocked and all others are, this will not lead to a deadlock. In this case, we added a new goroutine at the end that infinitely increments the variable `a`.

Conclusion

Great, we have completed one task from the live coding session. Next, you can expect even more interesting questions about Go.

Stay tuned for future posts in this series :)

Written by Nina Pakshina
Follow
957 followers · 3 following
Go developer in an online retail company. LinkedIn: https://www.linkedin.com/in/nina-pakshina-228156114/ Buy me a coffee: https://www.buymeacoffee.com/ninucium

Responses (12)

Write a response

What are your thoughts?

Shashank Pandey
Apr 3, 2024

Hey Nina, thanks for sharing such an insightful blog! It's been really helpful.

I just wanted to add a quick note for anyone reading this in the future. With the release of Golang 1.22, there's an important change to be aware of: now, Golang creates... more

97 2 replies Reply

Hima Bindu
Oct 5, 2024 (edited)

Hey thanks for the blog!

6 Reply

Excrispy
Jul 9, 2024 (edited)

The last example actually does block cause the channel is not closed. I tested it in a go playground.

But very helpful blog, thanks so much!

1 Reply

See all responses